

Automotive Marketplace

A multi-seller used-car marketplace website that installs on phones as an app

IN ONE SENTENCE

Dealers and private sellers list cars; buyers search, save and message; moderators keep it clean.

BUILT WITH

Node 22, Fastify 5, PostgreSQL 16, React 19, TanStack Query, Vite, Docker

What this guide covers

- 1 What the project is
- 2 The business problem it solves
- 3 The client's requirements
- 4 How the solution is designed
- 5 How to build it from scratch, step by step
- 6 Testing, deployment and known limits

Contents

1. What this project is	3
2. The business problem	4
3. Client requirements	5
4. How the solution is designed	8
5. Build it from scratch, step by step	10
6. Testing, deployment and known limits	15

1. What this project is

Automotive Marketplace is a website where **many sellers** list used cars for sale and **buyers** search, compare, save cars and message sellers. It is the kind of site people know from AutoTrader, PakWheels or Cars.com, sized for a regional marketplace or a dealer group that wants its own portal.

Sellers are either **dealers** (a business with a team, a phone number and a public dealer page) or **private sellers** (one person selling their own car). **Moderators** review new listings and handle reports, because the biggest threat to a car marketplace is scams. The site is a **Progressive Web App (PWA)**: it can be installed on a phone's home screen, has a phone-style tab bar, and still shows recently viewed cars without a signal.

At a glance	
Type	Public website + seller tools + moderation console, one web app
Users	Visitors, buyers, private sellers, dealer teams, moderators, admins
Stack	Node 22, Fastify 5, PostgreSQL 16, React 19, TanStack Query, Vite, Docker
Size of demo	2 dealers, 5 private sellers, ~50 cars with pictures, conversations, a scam in the review queue
Tests	15 shared unit tests, 34 API tests on a real database, 14 web tests, 40-check browser smoke test

Who uses it

User	What they do with it
Visitor	Searches and filters cars, opens car pages, shares links. No account needed.
Buyer (member)	Saves cars and searches, gets a count of new matches, messages sellers, reports bad listings.
Private seller	Lists up to 3 cars with photos, answers buyers, marks cars sold, renews listings.
Dealer team	Owner and staff share the dealer's stock, messages and CSV export; dealer page with phone number.
Moderator	Approves or sends back listings, handles reports, verifies dealers, suspends members.
Admin	Everything a moderator does, plus site settings, staff roles and the activity log.

2. The business problem

Used cars are sold through a patchwork of classified sites, social media groups and dealer websites. Buyers waste time on listings that are already sold, badly described or outright fraudulent; honest sellers compete with scammers; and a regional operator or dealer group has no marketplace of its own to build a brand on.

The client wants a **trusted local marketplace**: easy for sellers to list on, pleasant for buyers to search on a phone, and actively policed so that deposit scams and fake listings are caught before buyers see them.

Pain point today	What it costs the business	How this project solves it
Scam listings ("pay a deposit, car is with my shipping agent")	Buyers lose money, the site's reputation dies	Automatic warning signs send suspicious listings to a moderator; 3 reports take a listing down
Sellers post phone numbers and move buyers off-site	No record of conversations, fraud is untraceable	Built-in messaging; contact details in adverts are flagged
Listings stay up after the car is sold	Buyers waste time, trust drops	Listings expire after 60 days unless renewed; sold cars leave search
Search is slow or clumsy on phones	Most car shoppers browse on phones and leave	Phone layout, tab bar, installable app, offline car pages
Dealers can't share one account across the team	Messages get lost when staff change	Dealer accounts with a team; access follows team membership
Buyers must keep checking for new cars	They go to a competitor that alerts them	Saved searches count new matches since the last visit

Value to the business

- A marketplace that is **safe by design**, which is what makes buyers come back and sellers pay later (featured listings, dealer subscriptions).
- Dealers get a free shop window and a CSV of their stock performance (views, saves, messages per car).
- The operator gets moderation tools and a full activity log instead of policing by hand.

3. Client requirements

The brief below is written the way a client would hand it to a developer: what the system must do (functional requirements), how well it must do it (non-functional requirements), and what was explicitly left out. Every item below is implemented in the repository; section 6 explains how the project is tested.

Functional requirements

ID	Area	Requirement
FR-01	Search	Search by keyword, make and model (showing only makes and models on the site, with counts), price, year, mileage, fuel, gearbox, body type, town and seller type.
FR-02	Search	Sort by newest, price (both ways), lowest mileage and newest car; paging of 24 results with no duplicates between pages.
FR-03	Search	Every search is a shareable URL; search results show the first photo, price, price cut, key specs, town and seller.
FR-04	Car page	Photo gallery with next/previous and thumbnails; key facts; description; reference number.
FR-05	Car page	Seller card: dealers show name, verified badge, phone and a link to their stock; private sellers show first name only.
FR-06	Car page	Buyers can save the car, share it, message the seller and report the listing.
FR-07	Accounts	Anyone can sign up with name, email and password and is signed in straight away.
FR-08	Accounts	Members can change their password; sessions expire after inactivity.
FR-09	Selling	A listing starts as a draft; the seller adds details and up to 12 photos, reorders them and submits.
FR-10	Selling	A listing needs at least one photo and a 30-character description before it can be submitted.
FR-11	Selling	Sellers see status (draft, waiting for review, live, needs changes, sold, withdrawn, removed, expired), views, saves and conversations per car.
FR-12	Selling	Sellers can edit, mark sold, withdraw and relist, renew near expiry, delete drafts and export their listings as CSV.
FR-13	Selling	Private sellers may have a limited number of cars listed at once (default 3).
FR-14	Dealers	A member can open a dealer account (name, town, phone, website, about) and becomes its owner.
FR-15	Dealers	The owner adds and removes staff by email; everyone on the team manages the dealer's listings and messages.
FR-16	Dealers	Each dealer has a public page listing its cars.
FR-17	Buyers	Save cars (heart) and see them on a Saved page, including sold ones for 30 days.
FR-18	Buyers	Save searches and see how many new cars match since last looked; a badge shows the total.
FR-19	Buyers	Message sellers: one conversation per buyer and car, unread counts on both sides, email never shown.
FR-20	Buyers	Report a listing with a reason (scam, wrong details, sold, offensive, duplicate, other).

ID	Area	Requirement
FR-21	Moderation	Listings from private sellers and unverified dealers wait in a review queue with the seller's history.
FR-22	Moderation	Automatic warning signs always send a listing to review: phone/email/links in the description, scam payment wording, a price under half the median of comparable cars, no photos.
FR-23	Moderation	Moderators approve, or send back with a reason the seller sees; they can remove any listing permanently.
FR-24	Moderation	Reports from 3 different members take a listing off the site until a moderator decides.
FR-25	Moderation	Moderators verify or suspend dealers and suspend or restore members; admins change roles and site settings.
FR-26	Phone app	Installable on phones (manifest, icons, shortcuts); bottom tab bar on small screens.
FR-27	Phone app	Recently viewed search results, car pages and photos available offline; private data never cached.

Non-functional requirements

ID	Quality	Requirement
NFR-01	Security	Passwords hashed with bcrypt; accounts lock after 5 wrong attempts; session tokens stored only as hashes.
NFR-02	Security	Every change needs a CSRF token and a same-site Origin; strict Content-Security-Policy.
NFR-03	Security	Uploads are checked by their real bytes (JPEG/PNG/WebP), size-limited and counted; photos of hidden listings only served to owner and staff.
NFR-04	Security	Other people's listings, photos and conversations answer 404, never 403, so their existence is not revealed.
NFR-05	Abuse limits	Rate limits per signed-in session (per IP for visitors); 5 sign-ups per hour per address; 20 new conversations per member per day; 30 messages per minute; 10 reports per hour.
NFR-06	Integrity	Two people editing the same listing cannot overwrite each other (optimistic locking).
NFR-07	Integrity	Limits (private-seller cap, saved-search cap, daily contacts) cannot be passed by two quick clicks (row locks).
NFR-08	Integrity	One conversation per buyer and car, one open report per person and listing (database constraints).
NFR-09	Performance	Search uses indexes (full-text GIN index, partial indexes on live listings); photos are resized in the browser to 1600 px with a 480 px thumbnail before upload.
NFR-10	Performance	Public photos are cacheable by browsers and CDNs for a day.
NFR-11	Usability	Works on a 390 px phone with no sideways scrolling; dark mode; keyboard accessible dialogs and gallery.
NFR-12	Operations	One Docker image, health checks (/healthz , /readyz), migrations on start-up, structured logs with request ids, activity log of every business action.
NFR-13	Operations	Search engines get robots.txt and a sitemap.xml of live cars and dealer pages.

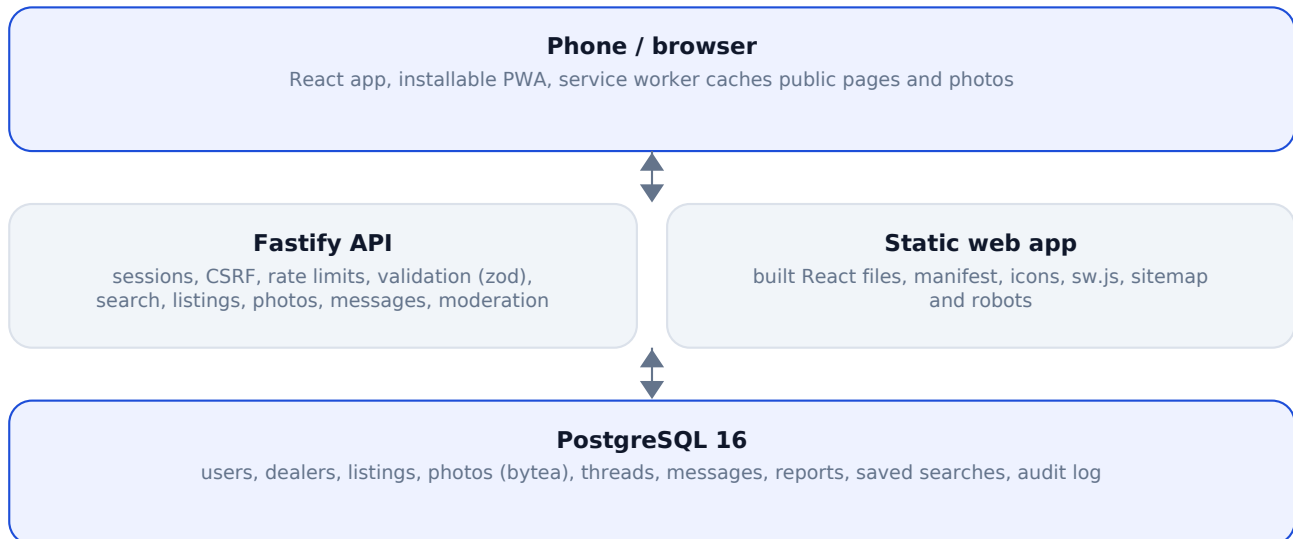
Out of scope for this release

- Email sending (verification, password reset by email, message alerts) — in-app badges instead.
- Payments, featured listings and dealer subscriptions.
- Distance search by postcode (towns are searched by name).
- Offline actions (sending messages or listing a car needs a connection).

4. How the solution is designed

One Node.js server serves both the API (`/api/...`) and the built React app. PostgreSQL holds everything, including photos, so one backup is complete. Rules that both sides need (input validation, warning signs, search filters) live in a `shared/` folder used by the server and the browser.

Architecture



Technology choices

Part	Choice	Why
Server	Node 22 + Fastify 5	Fast, small, first-class TypeScript; plugins for cookies, Helmet, rate limits, static files.
Database	PostgreSQL 16	Transactions and row locks for limits, constraints for integrity, built-in full-text search.
Validation	zod (shared)	One schema validates on the server and gives instant feedback in the browser.
Web app	React 19 + React Router + TanStack Query	Component UI, URL-driven search, caching and background refresh of data.
Build	Vite (web), esbuild (server)	Quick builds; the server bundles into a few files for the Docker image.
Tests	Vitest, Playwright (Chromium)	Unit and API tests on a real database, plus a browser run of the whole journey.
Delivery	Docker, docker-compose, GitHub Actions	Same image in CI and production; CI runs every test and boots the image.

Data model

Table / file	What it holds
users	Members, moderators and admins; password hash, lockout counters, optional dealer and dealer role.
dealers	Business name, slug for the public page, phone, town, verified and active flags.
sessions	Hashed session tokens with CSRF token, expiry and last-seen time.

Table / file	What it holds
listings	The car, price and previous price, status, warning flags, moderator note, views, publish and expiry dates, version; generated full-text search column.
photos	Image bytes and thumbnail per listing, in display order.
favourites / saved_searches	Saved cars; saved filters with the time the member last looked.
threads / messages	One conversation per buyer and listing, read times for each side; the messages.
reports	Reason, note and status; one open report per person and listing.
settings / audit_log	Site name, currency, limits; who did what, when and from where.

Key business rules

- **Who manages a listing:** a private listing belongs to its seller; a dealer listing belongs to whoever is on the dealer's team **now** (not the person who typed it in).
- **Review:** needs review = any warning sign, or (review setting on and the seller is not a verified dealer). An approved listing that is edited into something suspicious, or gets new photos from a seller who needs review, goes back to the queue.
- **Visible to the public** = status live, not expired, and the seller (or dealer) is not suspended. The same SQL fragment is used by search, car pages, photos, favourites and the sitemap.
- **Price cuts:** lowering the price of a live listing records the previous price, shown as "was £X"; a rise clears it.
- **Relisting** a withdrawn car counts as new: new publish date and expiry.

Folder layout

```
shared/      schemas.ts (zod rules) · market.ts (warning signs, filters) · types.ts
server/      src/app.ts · src/routes/*.ts · src/repo/*.ts · src/security/* · src/seed.ts
              migrations/001_init.sql · test/market.test.ts
web/         src/pages/*.tsx · src/components/* · src/lib/* · public/ (manifest, icons, sw.js)
              scripts/smoke.mjs
docs/        OPERATIONS.md · screenshots · PROJECT-GUIDE.pdf
Dockerfile · docker-compose.yml · .github/workflows/ci.yml
```

5. Build it from scratch, step by step

The steps below rebuild the project from an empty folder in the order that keeps it working at every stage: foundations first, then each feature end to end (database, API, test, screen), then the phone app, then delivery. Each step ends with a check you can run.

Before you start

- Node.js 22+, PostgreSQL 16 (local or Docker), Git, a code editor.
- Basic TypeScript, SQL and React. Chromium for the browser test (Playwright can download it).

Step 1. Set up the workspace

One repository with three packages that share code.

- 1.1 Create an npm workspace with `server` and `web` packages and a plain `shared` folder imported by both.

```
mkdir automotive-marketplace && cd automotive-marketplace && git init
npm init -y # add "workspaces": ["server", "web"]
npm create vite@latest web -- --template react-ts
mkdir -p server/src server/migrations shared
npm i -w server fastify @fastify/cookie @fastify/helmet @fastify/rate-limit @fastify/static
@fastify/compress pg zod
npm i -D -w server typescript tsx vitest esbuild @types/node @types/pg
```

- 1.2 Add TypeScript configs with `strict` on, and scripts: `dev`, `build`, `typecheck`, `test`, `migrate`, `seed:demo`.

Done when: `npm run typecheck` passes on the empty project.

Step 2. Configuration, database pool and migrations

- 2.1 Write `config.ts` that reads and validates environment variables with zod (`DATABASE_URL`, `PUBLIC_URL`, `COOKIE_SECURE`, `RATE_LIMIT_PER_MIN`, `MAX_PHOTO_BYTES`). Refuse to start on bad config.
- 2.2 Write `db.ts` (a `pg` pool with a statement timeout, and a `tx()` helper that commits or rolls back).
- 2.3 Write `migrate.ts`: apply `migrations/*.sql` in order, each in a transaction, recorded in `schema_migrations`, under an advisory lock so several servers can start together.

Done when: `npm run migrate` creates the `schema_migrations` table on a fresh database.

Step 3. Design the database schema

Put the business rules into the schema where the database can enforce them.

- 3.1 Create `users`, `sessions`, `settings`, `dealers`, `listings`, `photos`, `favourites`, `saved_searches`, `threads`, `messages`, `reports`, `audit_log` (see the data model in section 4).
- 3.2 Add CHECK constraints (status values, non-negative price and mileage), UNIQUE (listing, buyer) on threads, and a partial unique index for one open report per person and listing.
- 3.3 Add a generated `tsvector` search column with a GIN index, and partial indexes on live listings for date and price.

```
search tsvector GENERATED ALWAYS AS (  
  to_tsvector('simple', make || ' ' || model || ' ' || variant || ' ' || colour || ' ' ||  
  city)) STORED  
CREATE INDEX listings_search_idx ON listings USING gin (search);  
CREATE UNIQUE INDEX reports_open_once ON reports (listing_id, reporter_id) WHERE status =  
  'open';
```

Done when: The migration applies cleanly and re-running it does nothing.

Step 4. Security foundations: passwords, sessions, CSRF

- 4.1 Hash passwords with Node's built-in `scrypt`; compare in constant time; use a dummy hash for unknown emails so timing reveals nothing.
- 4.2 Sessions: random 32-byte token in an `httpOnly`, `SameSite=Lax` cookie; store only its SHA-256 plus a CSRF token, expiry and last-seen time.
- 4.3 In an `onRequest` hook: read the session for `/api/*`, reject writes whose Origin is another site, and require the `x-csrf-token` header on every write.
- 4.4 Register Helmet (CSP: scripts and images from self, `blob:` for photo previews) and the rate limiter keyed by session id when signed in, otherwise by IP.
- 4.5 Routes: `POST /api/auth/register`, `login` (lock after 5 failures), `logout`, `me`, `password`.

Done when: An API test can register, sign in, call `/api/auth/me`, and a write without the CSRF header gets 403.

Step 5. Shared rules: schemas, filters and warning signs

- 5.1 In `shared/schemas.ts`, define zod schemas for sign-up, listings, search filters (empty values ignored, enums checked, page capped at 100), messages, reports and moderation decisions.
- 5.2 In `shared/market.ts`, write `reviewFlags()` (regexes for emails, phone numbers, links and scam wording; price below half the comparable median; no photos) and `submitProblems()`.
- 5.3 Unit-test them, including the tricky cases: mileage like `45,000` and engine sizes like `1.6` must **not** count as phone numbers.

Done when: `vitest run --root shared` passes.

Step 6. Listings for sellers: draft, edit, submit

- 6.1 `POST /api/my/listings` creates a draft (owned by the member, or by their dealer if they belong to one).
- 6.2 `PUT /api/my/listings/:id` updates with the `version` the form loaded; zero rows updated means someone else changed it → 409.
- 6.3 `POST ../submit` checks photos and description, the private-seller cap (under a lock on the user row), computes warning signs, then sets `pending` or goes live with publish and expiry dates.
- 6.4 Add `sold`, `withdraw`, `renew` and draft `DELETE`, each as a guarded status change inside a transaction, and write every action to `audit_log`.

Done when: API tests cover draft → pending → approved, a stale edit returning 409, and the cap.

Step 7. Photo uploads

- 7.1 Accept raw `image/jpeg|png|webp` bodies up to `MAX_PHOTO_BYTES`; check the first bytes of the file (magic numbers), not the header.
- 7.2 Store bytes in `photos`, max 12 per listing; separate endpoint for a thumbnail; endpoints to reorder and delete.
- 7.3 Serve `GET /api/photos/:id` only when the listing is public, or the viewer owns it or is staff; public photos get `cache-control: public, max-age=86400`.
- 7.4 In the browser, shrink each photo with a canvas to 1600 px and 480 px JPEGs before uploading (this also strips GPS data).

Done when: Tests: a fake image is refused, a 13th photo is refused, a hidden listing's photo is 404 for the public.

Step 8. Public search and car pages

- 8.1 Write `searchWhere()` that turns filters into a WHERE clause with bound parameters only; free text uses `websearch_to_tsquery`; % and _ typed by users are escaped.
- 8.2 `GET /api/listings` returns 24 cards, the total, pages, and make counts (ignoring the make filter so other makes stay selectable). Every sort ends on the id so pages never overlap.
- 8.3 `GET /api/listings/:id` returns the car, photos and seller card; counts a view for public live listings; sold cars stay visible for 30 days.
- 8.4 Build the React search page: filters in the URL, sort select, pager, car cards with lazy thumbnails, and a car page with gallery, facts, seller card and contact box.

Done when: Filtering, sorting and paging work in the browser and in API tests, including an SQL-injection attempt in the keyword box.

Step 9. Dealers and teams

- 9.1 `POST /api/dealer` creates a dealer with a unique slug (`smith-motors`, `smith-motors-2`) and makes the member its owner.
- 9.2 Owner adds staff by email and removes them; staff can leave. Access is checked as: dealer listing → current team only.
- 9.3 Public `GET /api/dealers/:slug` page with the dealer's live cars.

Done when: Test: a removed staff member loses access even to cars they listed themselves.

Step 10. Buyers: saved cars, saved searches, messages, reports

- 10.1 Favourites: `PUT/DELETE /api/saved/:id` (only public listings can be saved).
- 10.2 Saved searches store cleaned filters and `last_seen_at`; the list counts matches published since then; "Show cars" marks them seen.
- 10.3 Messages: `POST /api/listings/:id/messages` finds or creates the one thread for this buyer and car (daily limit on new threads), `GET /api/threads` with unread counts per side, reply endpoint.
- 10.4 Reports: one open report per person; when 3 different members have open reports on a non-verified seller's listing, move it to `pending` automatically.
- 10.5 `GET /api/unread` feeds the badges (messages, new matches, queue size for staff).

Done when: Tests cover one thread per car, unread counts, the daily limit and the three-report hold.

Step 11. Moderation console and admin

- 11.1 Queue: pending listings oldest first, with photos, warning signs and the seller's history.
- 11.2 Approve (go live, dismiss open reports) or send back with a reason; remove permanently (closes all reports).
- 11.3 Reports list with dismiss / remove; dealers list with verified and active switches; members list with suspend / restore (sessions deleted).
- 11.4 Admin page for site settings (name, currency, listing days, limits) and the activity log; only admins change roles.

Done when: Role tests: members get 403 on staff routes; moderators cannot change roles.

Step 12. Make it a phone app (PWA)

- 12.1 Add `manifest.webmanifest` (standalone display, theme colour, 192/512 px and maskable icons, shortcuts) and link it from `index.html`.
- 12.2 Write `sw.js`: pages network-first falling back to the cached app shell; hashed assets cache-first; public API reads network-first with a cached copy; photos cache-first with a size cap. Never cache private endpoints; clear the API cache on sign-out.
- 12.3 Register the service worker in production builds only; add an "Install app" button using `beforeinstallprompt`, an offline banner, and a bottom tab bar under 760 px.

Done when: In Chrome DevTools → Application the manifest is valid and the worker is active; with the network off, a car you viewed still opens.

Step 13. Demo data

- 13.1 Write `seed.ts`: dealers, private sellers, a buyer, staff, ~50 cars with prices from a depreciation formula, favourites, saved searches, conversations, a report and a review queue that includes an obvious scam.
- 13.2 Draw demo car pictures in code and encode them as PNG, so the demo needs no image files or internet.
- 13.3 Guard it: refuse in production unless `ALLOW_DEMO_SEED=1`.

Done when: `npm run seed:demo` then `npm run dev` shows a populated site; every demo login works.

Step 14. Test the whole thing

- 14.1 API tests (Vitest) run against a real PostgreSQL database that the test set-up drops and recreates (only if its name contains `test`).
- 14.2 Browser smoke test (`web/scripts/smoke.mjs`, Playwright): build, start the real server on a fresh database, then drive a visitor, a new seller uploading real photos, a moderator, a buyer and the seller replying, then the phone layout and offline mode. Fail on any script error.

```
npm run typecheck && npm test
npm run build && npm run smoke
```

Done when: All suites green locally.

Step 15. Package and deliver

- 15.1 Multi-stage `Dockerfile` (build, then a slim runtime with production dependencies only, non-root user, health check).
- 15.2 `docker-compose.yml` with PostgreSQL and the app; `.env.example` documenting every setting.
- 15.3 GitHub Actions: typecheck, tests with a PostgreSQL service, build, smoke test, dependency audit, and a job that builds the image and checks it boots.
- 15.4 Write `README.md` and `docs/OPERATIONS.md` (deploy, https, backups, photos, moderation routines, limits).

Done when: CI is green on GitHub and `docker compose up` serves a working site.

6. Testing, deployment and known limits

How it is tested

- **15 shared unit tests:** warning signs (including false-positive cases), filter parsing, listing validation.
- **34 API tests on PostgreSQL:** search, paging and injection attempts; hidden listings and photos; sign-up rate limit and CSRF; draft → review → live; upload checks; private-seller cap; verified dealers and warning signs; price cuts and stale edits; dealer teams; conversations and limits; saved searches; three-report hold; suspensions; roles; sold, expiry, renewal and relisting; CSV injection; per-session rate limits.
- **14 web unit tests:** prices, dates, photo sizing, search URLs.
- **40-check browser smoke test** of the whole journey, on desktop and a 390 px phone, including offline mode. CI runs it with a current Chromium; it caught a real crash (a React effect returning a Promise in newer Chrome) that older browsers hid.

```
npm run typecheck
npm test           # shared, API (needs PostgreSQL) and web tests
npm run build && npm run smoke
```

Deploying and running it

```
cp .env.example .env           # set POSTGRES_PASSWORD and PUBLIC_URL
docker compose up -d --build
docker compose exec app node dist/cli/create-admin.js --email you@example.com --name "Your Name"
# optional demo: docker compose exec -e ALLOW_DEMO_SEED=1 app node dist/cli/seed-demo.js
```

- Put Caddy or nginx in front for https (a PWA needs https), set `TRUST_PROXY=true` behind a proxy, and turn on database backups. Details are in [docs/OPERATIONS.md](#).

Known limits

- No email yet (verification, reset by email, message alerts).
- Town search by name, not distance from a postcode.
- Photos are stored in PostgreSQL; move them to object storage (S3, R2) at scale.
- The phone-number warning is cautious: a list of service years can trip it, and a moderator simply approves.
- Rate limits are counted per server instance (use Redis for several instances).

Sensible next steps

- Email with a provider such as Resend or Postmark (verification, reset, message and saved-search alerts).
- A price guide on each car using the used-car price estimator project.
- Featured listings and dealer subscriptions (payments).
- Feed dealer stock in automatically from the dealer management system via the dealership-automation project.